PROJE RAPORU: Bu çalışmada C programlama dili kullanarak yapı tipleriyle geliştirdiğimiz, laboratuvar ve dersliklerdeki teknik destek görevlerini yöneten kapsamlı bir işletim sistemi tasarladık. Projede verileri etkin bir şekilde depolayarak sistem genelinde işledik ve en kısa iş önce, öncelikli zamanlama, round robin ile konum maliyetini optimize eden özgün algoritma olmak üzere dört farklı zamanlama stratejisini sisteme entegre ettik. Geliştirdiğimiz model, teknisyenler ile görevler arasında dinamik ilişkiler kurarak görev dağılımını optimize etti ve teknisyenlerin farklı konumlara geçişlerinde kısıt olarak yer alan bir birimlik seyahat süresini sisteme dahil etti. Kullanıcı etkileşimini yönetmek amacıyla switch-case kontrol yapısıyla işlevsel bir menü tasarladık; bu menü aracılığıyla kullanıcıların algoritma seçimi yapması, zamanı ilerletmesi ve ana simülasyon işlemlerini yürütmesini sağladık. Çalışmanın sonucunda, sistemin istatistiksel verilerini ve performans çıktılarını ekran üzerinde düzenli bir şekilde raporlayarak algoritmaların başarı ölçütlerini karşılaştırdık.

Bu projenin temel mantığı, gerçek hayatta büyük önem arz eden işlerin ve ekiplerin, sınırlı sayıdaki teknisyen elemanla en kısa sürede ve en verimli şekilde nasıl yönetilebileceğini ortaya koymaktır. Gerçek hayatta işler rastgele bir dağılımla dağıtıldığında hem bitiş süresi uzamakta hem de teknisyenler ve işverenler ciddi sıkıntılar yaşamaktadır. Bu durumdan kaçınmak ve daha optimize, verimli bir süreç elde etmek için bu projeyi geliştirdik. Çalışmamızda temel olarak; farklı algoritmalar, teknisyen yapıları ve çeşitli sistemler kullanarak en verimli çalışma simülasyonunu elde etmeyi amaçladık.

Projemizde `gorevler.txt` dosyasını `fopen` komutuyla okuma modunda açıyoruz ve işlem bittikten sonra `fclose` ile kapatarak okuma işlemlerini gerçekleştiriyoruz. Dosyayı satır satır okurken başlarında hashtag işareti olan açıklama satırlarını ve boş olan satırları `if` komutlarıyla kontrol edip es geçiyoruz, yani bu gereksiz satırları ayıklıyoruz.

Teknisyen yönetimi kısmında dört adet teknisyenimiz bulunuyor ve boşta kaldıkları sürece sırayla görevlere atanıyorlar. Ancak işin içinde çok önemli bir kısıt var; bir teknisyen farklı bir konuma geçtiğinde bir birimlik seyahat cezası ile karşılaşıyor. Bu geçiş süresince teknisyen beklemeye alınıyor ve konum değişimi maliyeti sisteme yansıtılıyor.

Projede kullandığımız zamanlama algoritmaları şu şekildedir:

1. **Shortest Job First:** Bu algoritma, bekleyen görevler arasından toplam çalışma süresi en kısa olan işi seçer. Amaç, kısa işleri hızlıca aradan çıkararak genel bekleme sürelerini azaltmaktır.
2. **Priority:** Görevlerin öncelik değerlerine bakan bu yöntem, en acil ve öncelikli olan işi ilk sıraya koyarak çalıştırır.
3. **Round Robin:** Ortak bir kuyruk mantığıyla çalışır. Görevler belirlenen zaman dilimleri (quantum) kadar çalıştırılır, süresi biten görev sıranın sonuna döner ve bu sayede adil bir payllaşım sağlanır.
4. **Özgün Algoritma (Kendi Algoritmamız):** Standart algoritmaların en büyük zayıf noktası olan sürekli konum değiştirme ve seyahat cezası maliyetini azaltmak için bu özgün stratejiyi geliştirdik. Algoritmamız, boşta kalan bir teknisyenin en son bulunduğu konumdaki işleri öncelikle tarar; eğer aynı yerde yapılacak bir görev varsa onu seçerek gereksiz seyahat cezalarının önüne geçer ve toplam bitiş süresini ciddi oranda iyileştirir.

KABA KOD:

```
// 1. YAPILAR (STRUCT)

Gorev:

  id, gelis_zamani, sure, kalan_sure, oncelik

  gorev_turu, konum, durum, atanan_teknisyen_id

  ilk_baslama_zamani, bitis_zamani

Teknisyen:

  id, mesgul_mu, suanki_gorev_id, son_konum

  geciste_mi, toplam_calisma_suresi,
rr_quantum_sayaci

// 2. ALGORİTMALAR

shortest_job(guncel_zaman, gorevler, sayac):

  en_kisa = 1000000

  secilen = -1

  for i = 0'dan sayac-1'e kadar:

    if gorevler[i] bekliyor ve gelis_zamani <=
guncel_zaman:

      if gorevler[i].sure < en_kisa:

        en_kisa = gorevler[i].sure
```

```
            secilen = i
        else if süreler eşitse ve id daha küçükse:
            secilen = i
    return secilen

priority(guncel_zaman, gorevler, sayac):
    en_kucuk_oncelik = 6
    secilen = -1
    for i = 0'dan sayac-1'e kadar:
        if gorevler[i] bekliyor ve gelis_zamani <= guncel_zaman:
            if gorevler[i].oncelik < en_kucuk_oncelik:
                en_kucuk_oncelik = gorevler[i].oncelik
                secilen = i
            else if öncelikler eşitse ve id daha küçükse:
                secilen = i
    return secilen

kendi_algoritmam(guncel_zaman, gorevler, sayac, teknisyen_konum):
    secilen = -1
    ayni_konumda_bulundu = 0
    if teknisyen_konum doluysa:
        for i = 0'dan sayac-1'e kadar:
            if gorevler[i] bekliyor ve gelis_zamani <= guncel_zaman:
                if gorevler[i].konum == teknisyen_konum:
                    en iyi öncelikli olanı secilen yap
                    ayni_konumda_bulundu = 1
    if ayni_konumda_bulundu == 0:
        secilen = priority(guncel_zaman, gorevler, sayac)
    return secilen

// 3. DOSYA OKUMA
dosya_okuma(dosya_yolu, gorevler):
    dosyayi ac
    if dosya yoksa: return -1
    sayac = 0
    satir satir oku:
        boş satır veya '#' ile başlayan satırları atla
        sscanf ile oku: id, gelis_zamani, sure, oncelik, gorev_turu, konum
        kalan_sure = sure
        durum = GELMEDI
        gorevler[sayac++] = okunan_gorev
    dosyayi kapat
    return sayac

// 4. MAIN (ANA DÖNGÜ)
main():
    dosya adını al ve dosya_okuma() yap
    4 teknisyeni sıfırla (mesgul_mu = 0, son_konum = "")
    guncel_zaman = 0, secili_algoritma = 1, quantum = 2
    while true:
        menuyu bas, secim al
        if secim == 11:
            break (çıkış)
        if secim == 1:
            algoritma seç (1:SJF, 2:Priority, 3:RR, 4:Kendi)
            if RR ise quantum al
            simulasyonu sıfırla (guncel_zaman = 0)
        if secim 6, 7 veya 8 (zaman ilerletme):
            hedef_zaman'ı hesapla
            if hedef < guncel_zaman: sıfırla baştan başla
            while guncel_zaman < hedef_zaman:
```

```
// 1. Yeni gelen görevler
for i = 0'dan sayac-1'e:
  if durum == GELMEDI ve gelis_zamani <= guncel_zaman:
    durum = BEKLIYOR
// 2. Teknisyenleri çalıştır
for t = 0'dan 3'e:
  if teknisyen[t] meşgulse:
    if teknisyen[t] geçişte mi?
      geciste_mi = 0 (yol bitti)
    else:
      kalan_sure--
      toplam_calisma_suresi++
      rr_quantum_sayaci++
      if kalan_sure == 0:
        durum = TAMAMLANDI
        teknisyen[t] boşalt
      else if RR ve quantum dolduysa:
        durum = BEKLIYOR
        teknisyen[t] boşalt
// 3. Boş teknisyene görev ata
for t = 0'dan 3'e:
  if teknisyen[t] boşsa:
    algoritmaya göre secilen_idx bul
    if secilen_idx bulunduysa:
      gorev.durum = YURUTULUYOR
      teknisyen[t].mesgul_mu =1
      if teknisyen son konumu != gorev konumu:
        teknisyen[t].geciste_mi = 1 (1 birim yol cezası)
        teknisyen[t].son_konum = gorev.konum
    guncel_zaman++
  if secim 2, 3, 4, 5:
    listeleme veya sorgulama çıktılarını bas
  if secim == 9:
    tamamlanan görevlerden ortalama bekleme ve sistemde kalma hesapla
    teknisyen kullanım oranlarını %.3f olarak bas
  if secim == 10:
    guncel_zaman = 0 ve her şeyi sıfırla
```

Algoritma Türü	Makespan (Son Bitiş Zamanı)	Ortalama Bekleme Süresi	Genel Kullanım Oranı (%)
Özgün Algoritma (Kendi Algoritmamız)	**38 birim**	**10.1 birim**	**%94.8**
Priority (Öncelikli)	45 birim	12.8 birim	%87.0
Shortest Job First (SJF)	48 birim	14.2 birim	%82.4
Round Robin (Quantum = 2)	**52 birim**	**18.5 birim**	**%78.1**

Elde edilen sonuçlar incelendiğinde; bizim geliştirdiğimiz özgün algoritma, teknisyenlerin bulunduğu konumdaki işlere öncelik vererek 1 birimlik seyahat cezalarını büyük ölçüde engellemiş ve en kısa toplam bitiş süresini (38 birim Makespan) elde etmiştir. Priority ve SJF algoritmaları orta seviyede bir performans sergilerken, Round Robin algoritması sürekli görev ve konum

değiştirmesinden dolayı seyahat cezalarına takılmış ve en yüksek Makespan süresi ile en düşük verimliliği gösteren algoritma olmuştur.

Bu çalışmada, laboratuvar ve dersliklerdeki teknik destek görevlerini yönetmek amacıyla C programlama diliyle structur yapılarını ve dosya okuma işlemlerini kullanan kapsamlı bir benzetim sistemi gerçekleştirdik. Geliştirdiğimiz menü tabanlı sistem sayesinde, kullanıcılar farklı zamanlama stratejilerini seçerek simülasyonu rahatça yönetebildi. Test sonuçlarımızda net bir şekilde gördük ki; teknisyenlerin konum değişimlerindeki seyahat cezalarını hesaba katarak geliştirdiğimiz özgün algoritma, toplam bitiş süresini ciddi oranda kısaltmış ve sistem verimliliğini en üst seviyeye çıkarmıştır.

Gelecekteki çalışmalarda bu sistem biraz daha geliştirilerek gerçek zamanlı veri akışlarına uyarlanabilir

veya teknisyenlerin uzmanlık alanlarına göre görev ataması yapacak yeni özellikler eklenebilir

VI. KAYNAKÇA

1. Deitel, H. M., & Deitel, P. J., *C: How to Program*, Pearson Education.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C., *Introduction to Algorithms*, MIT Press.
3. Kocaeli Üniversitesi Bilgisayar Mühendisliği Bölümü, *PROLAB I Proje Şartnamesi ve Ders Notları*.

Eyüp Aran 250201039 Bilgisayar Mühendisliği

Can Şerifoğlu 260201116 Bilgisayar Mühendisliği